

Especificación Técnica: Sistema de Completitud de Perfil

Documento de Implementación para Claude Code

Versión: 1.0

Fecha: Mayo 2026

Producto: Resumint

Objetivo: Definir el algoritmo exacto de cálculo de completitud del perfil de usuario

1. Contexto y Propósito

El sistema de completitud de perfil es un componente crítico de Resumint que:

1. Calcula un puntaje numérico (0-100) que representa qué tan completo está el perfil del usuario
 2. Determina si el usuario puede o no generar un CV
 3. Orienta al usuario sobre qué información falta
 4. Se recalcula en tiempo real tras cada operación que modifique el perfil
-

2. Estructura de Datos del Perfil

2.1 Esquema TypeScript de Referencia

```
interface PerfilUsuario {
```

```
  // Capa 1 - Información personal (onboarding)
```

```
  informacion_personal: {
```

```
    nombre: string;
```

```
    apellido: string;
```

```
    foto_url?: string | null;
```

```
telefono?: string | null;

ciudad?: string | null;

pais?: string | null;

email_cv: string;

profesion_perfil?: string | null;

};

// Capa 2 - Perfil profesional

profesiones_inferidas: string[];

resumen_profesional?: string | null; // NO SE USA PARA COMPLETITUD

experiencia: ExperienciaLaboral[];

educacion: Educacion[];

habilidades: string[];

logros: string[];

idiomas: Idioma[];


// Metadata

puntaje_completitud: number; // 0-100

ultima_actualizacion: Date;

}

interface ExperienciaLaboral {

    id: string;
```

```
cargo: string;

empresa: string;

fecha_inicio: Date;

fecha_fin?: Date | null;

descripcion?: string;

actual: boolean;

}
```

```
interface Educacion {

  id: string;

  titulo: string;

  institucion: string;

  fecha_inicio: Date;

  fecha_fin?: Date | null;

  en_curso: boolean;

}
```

```
interface Idioma {

  id: string;

  idioma: string;

  nivel: 'Básico' | 'Intermedio' | 'Avanzado' | 'Nativo';

}
```

3. Fórmula de Completitud

3.1 Resumen Ejecutivo

Total máximo: 100 puntos

- **Capa 1 (Datos básicos):** 30 puntos máximos
- **Capa 2 (Perfil profesional):** 70 puntos máximos

3.2 Capa 1 — Datos Básicos (30 puntos)

Campo(s)	Condición de Validación	Puntos
nombre Y apellido	Ambos presentes y no vacíos (trim)	10
email_cv	Presente, no vacío y válido como email	5
foto_url	URL presente y no null	5
profesion_perfil	String presente y no vacío (trim)	5
telefono Y (ciudad O pais)	Teléfono presente Y al menos uno de ciudad/país presente	5

Pseudocódigo Capa 1:

```
function calcularPuntosCapa1(info: InformacionPersonal): number {  
  
    let puntos = 0;  
  
    // Nombre + Apellido (10 puntos)  
  
    if (info.nombre?.trim() && info.apellido?.trim()) {  
  
        puntos += 10;  
  
    }  
}
```

```
// Email CV (5 puntos)
```

```
if (info.email_cv?.trim() && esEmailValido(info.email_cv)) {
```

```
    puntos += 5;
```

```
}
```

```
// Foto (5 puntos)
```

```
if (info.foto_url) {
```

```
    puntos += 5;
```

```
}
```

```
// Profesión (5 puntos)
```

```
if (info.profesion_perfil?.trim()) {
```

```
    puntos += 5;
```

```
}
```

```
// Teléfono + Ciudad/País (5 puntos)
```

```
if (info.telefono?.trim() && (info.ciudad?.trim() || info.pais?.trim())) {
```

```
    puntos += 5;
```

```
}
```

```
    return puntos;
}
```

3.3 Capa 2 — Perfil Profesional (70 puntos)

3.3.1 Experiencia Laboral (máximo 30 puntos)

Regla: 15 puntos por entrada, contabilizando máximo las primeras 2 entradas.

Validación de entrada:

- Debe tener **cargo** y **empresa** no vacíos
- Debe tener **fecha_inicio** válida

Pseudocódigo:

```
function calcularPuntosExperiencia(experiencias: ExperienciaLaboral[]): number {

    const experienciasValidas = experiencias.filter(exp =>

        exp.cargo?.trim() &&

        exp.empresa?.trim() &&

        exp.fecha_inicio

    );

    const cantidadContabilizada = Math.min(experienciasValidas.length, 2);

    return cantidadContabilizada * 15;

}
```

Casos:

- 0 experiencias válidas → 0 puntos
- 1 experiencia válida → 15 puntos
- 2+ experiencias válidas → 30 puntos

3.3.2 Educación (máximo 20 puntos)

Regla: 10 puntos por entrada, contabilizando máximo las primeras 2 entradas.

Validación de entrada:

- Debe tener **titulo** e **institucion** no vacíos
- Debe tener **fecha_inicio** válida

Pseudocódigo:

```
function calcularPuntosEducacion(educacion: Educacion[]): number {  
  
  const educacionValida = educacion.filter(edu =>  
  
    edu.titulo?.trim() &&  
  
    edu.institucion?.trim() &&  
  
    edu.fecha_inicio  
  
  );  
  
  
  const cantidadContabilizada = Math.min(educacionValida.length, 2);  
  
  return cantidadContabilizada * 10;  
  
}
```

Casos:

- 0 educaciones válidas → 0 puntos
- 1 educación válida → 10 puntos
- 2+ educaciones válidas → 20 puntos

3.3.3 Habilidades (máximo 7 puntos)

Regla escalonada:

- 0-2 habilidades → 0 puntos
- 3-5 habilidades → 5 puntos

- 6+ habilidades → 7 puntos

Validación de entrada:

- String no vacío después de trim

Pseudocódigo:

```
function calcularPuntosHabilidades(habilidades: string[]): number {  
  
    const habilidadesValidas = habilidades.filter(h => h?.trim());  
  
    const cantidad = habilidadesValidas.length;  
  
    if (cantidad >= 6) return 7;  
  
    if (cantidad >= 3) return 5;  
  
    return 0;  
}
```

Tabla de casos:

Cantidad de habilidades válidas	Puntos
0, 1, 2	0
3, 4, 5	5
6+	7

3.3.4 Logros (máximo 8 puntos)

Regla escalonada:

- 0 logros → 0 puntos
- 1-2 logros → 5 puntos
- 3+ logros → 8 puntos

Validación de entrada:

- String no vacío después de trim

Pseudocódigo:

```
function calcularPuntosLogros(logros: string[]): number {  
  
    const logrosValidos = logros.filter(l => l?.trim());  
  
    const cantidad = logrosValidos.length;  
  
    if (cantidad >= 3) return 8;  
  
    if (cantidad >= 1) return 5;  
  
    return 0;  
}
```

Tabla de casos:

Cantidad de logros válidos	Puntos
0	0
1, 2	5
3+	8

3.3.5 Idiomas (máximo 5 puntos)**Regla binaria:**

- 0 idiomas → 0 puntos
- 1+ idiomas → 5 puntos

Validación de entrada:

- Debe tener campo **idioma** no vacío

Pseudocódigo:

```
function calcularPuntosIdiomas(idiomas: Idioma[]): number {
```

```
const idiomasValidos = idiomas.filter(i => i.idioma?.trim());

return idiomasValidos.length >= 1 ? 5 : 0;

}
```

3.4 Función Principal de Cálculo

```
function calcularCompleitudPerfil(perfil: PerfilUsuario): number {

  const puntosCapa1 = calcularPuntosCapa1(perfil.informacion_personal);


  const puntosExperiencia = calcularPuntosExperiencia(perfil.experiencia);

  const puntosEducacion = calcularPuntosEducacion(perfil.educacion);

  const puntosHabilidades = calcularPuntosHabilidades(perfil.habilidades);

  const puntosLogros = calcularPuntosLogros(perfil.logros);

  const puntosIdiomas = calcularPuntosIdiomas(perfil.idiomas);


  const puntosCapa2 =

    puntosExperiencia +

    puntosEducacion +

    puntosHabilidades +

    puntosLogros +

    puntosIdiomas;


  const puntajeTotal = puntosCapa1 + puntosCapa2;
```

```
return Math.min(puntajeTotal, 100); // Seguridad: nunca exceder 100  
}
```

4. Lógica de Pérdida de Puntos al Eliminar Información

4.1 Principio General

Los puntos se restan SOLO si la eliminación causa que la categoría caiga por debajo del umbral de contabilización.

4.2 Casos por Categoría

4.2.1 Experiencia Laboral

Umbral de contabilización: Máximo 2 entradas válidas

Antes de eliminar	Después de eliminar	Cambio de puntos
1 experiencia (15 pts)	0 experiencias	-15 puntos
2 experiencias (30 pts)	1 experiencia	-15 puntos
3 experiencias (30 pts)	2 experiencias	0 puntos (sin cambio)
4+ experiencias (30 pts)	3+ experiencias	0 puntos (sin cambio)

Lógica:

```
// Antes de eliminar
```

```
const puntosAntes = calcularPuntosExperiencia(experiencias);
```

```
// Después de eliminar
```

```
const experienciasFiltradas = experiencias.filter(exp => exp.id !== idAEliminar);
```

```
const puntosDespues = calcularPuntosExperiencia(experienciasFiltradas);
```

const puntosPerdidos = puntosAntes - puntosDespues;

4.2.2 Educación

Umbral de contabilización: Máximo 2 entradas válidas

Antes de eliminar	Después de eliminar	Cambio de puntos
1 educación (10 pts)	0 educaciones	-10 puntos
2 educaciones (20 pts)	1 educación	-10 puntos
3 educaciones (20 pts)	2 educaciones	0 puntos (sin cambio)
4+ educaciones (20 pts)	3+ educaciones	0 puntos (sin cambio)

4.2.3 Habilidades

Umbrales: 3-5 = 5pts, 6+ = 7pts

Antes de eliminar	Después de eliminar	Cambio de puntos
2 habilidades (0 pts)	1 habilidad	0 puntos
3 habilidades (5 pts)	2 habilidades	-5 puntos
4 habilidades (5 pts)	3 habilidades	0 puntos
5 habilidades (5 pts)	4 habilidades	0 puntos
6 habilidades (7 pts)	5 habilidades	-2 puntos
7+ habilidades (7 pts)	6+ habilidades	0 puntos

4.2.4 Logros

Umbrales: 1-2 = 5pts, 3+ = 8pts

Antes de eliminar	Después de eliminar	Cambio de puntos
1 logro (5 pts)	0 logros	-5 puntos
2 logros (5 pts)	1 logro	0 puntos
3 logros (8 pts)	2 logros	-3 puntos

Antes de eliminar	Después de eliminar	Cambio de puntos
4+ logros (8 pts)	3+ logros	0 puntos

4.2.5 Idiomas

Umbral: 1+ = 5pts

Antes de eliminar	Después de eliminar	Cambio de puntos
1 idioma (5 pts)	0 idiomas	-5 puntos
2+ idiomas (5 pts)	1+ idiomas	0 puntos

4.3 Implementación Genérica

```
function actualizarCompleitudTrasEliminacion(
```

```
    perfil: PerfilUsuario,
```

```
    tipo: 'experiencia' | 'educacion' | 'habilidad' | 'logro' | 'idioma',
```

```
    idAEliminar: string
```

```
): number {
```

```
    // Calcular puntaje antes de eliminar
```

```
    const puntajeAntes = calcularCompleitudPerfil(perfil);
```

```
    // Crear copia del perfil sin el elemento eliminado
```

```
    const perfilActualizado = { ...perfil };
```

```
    switch(tipo) {
```

```
        case 'experiencia':
```

```
            perfilActualizado.experiencia = perfil.experiencia.filter(
```

```
    exp => exp.id !== idAEliminar

);

break;

case 'educacion':

    perfilActualizado.educacion = perfil.educacion.filter(

        edu => edu.id !== idAEliminar

    );

    break;

case 'habilidad':

    perfilActualizado.habilidades = perfil.habilidades.filter(

        (_, index) => index.toString() !== idAEliminar

    );

    break;

case 'logro':

    perfilActualizado.logros = perfil.logros.filter(

        (_, index) => index.toString() !== idAEliminar

    );

    break;

case 'idioma':

    perfilActualizado.idiomas = perfil.idiomas.filter(

        idioma => idioma.id !== idAEliminar
```

```
);  
  
break;  
  
}  
  
// Calcular nuevo puntaje  
  
const puntajeDespues = calcularCompleitudPerfil(perfilActualizado);  
  
return puntajeDespues;  
  
}
```

5. Triggers de Recálculo

El puntaje de completitud debe recalcularse automáticamente en los siguientes eventos:

5.1 Operaciones que Disparan Recálculo

Operación	Descripción
<code>onCreate</code>	Cuando se crea el perfil por primera vez
<code>onUpdate</code>	Cuando se actualiza cualquier campo de <code>informacion_personal</code>
<code>onAddExperiencia</code>	Cuando se agrega una experiencia laboral
<code>onUpdateExperiencia</code>	Cuando se edita una experiencia existente
<code>onDeleteExperiencia</code>	Cuando se elimina una experiencia
<code>onAddEducacion</code>	Cuando se agrega educación
<code>onUpdateEducacion</code>	Cuando se edita educación existente
<code>onDeleteEducacion</code>	Cuando se elimina educación

Operación	Descripción
onAddHabilidad	Cuando se agrega una habilidad
onDeleteHabilidad	Cuando se elimina una habilidad
onAddLogro	Cuando se agrega un logro
onDeleteLogro	Cuando se elimina un logro
onAddIdioma	Cuando se agrega un idioma
onDeleteIdioma	Cuando se elimina un idioma
onDocumentParsed	Cuando se procesa un documento y se integra al perfil

5.2 Implementación con Hooks (Ejemplo Supabase)

// Trigger en base de datos

```
CREATE OR REPLACE FUNCTION recalcular_completitud()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    NEW.puntaje_completitud := calcular_completitud_perfil(NEW);
```

```
    NEW.ultima_actualizacion := NOW();
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

// Aplicar a todas las tablas relevantes

```
CREATE TRIGGER trigger_recalcular_completitud
```

```
BEFORE INSERT OR UPDATE ON perfiles
```

```
FOR EACH ROW
```

EXECUTE FUNCTION recalculer_completitud();

6. Reglas de Negocio Basadas en Completitud

6.1 Bloqueo de Generación de CV

El mensaje UI aplica para la seccion Crear CV, especificamente para la tarjeta de completitud que aparece.

Rango de Completitud	Comportamiento	Mensaje UI
0% - 29%	 Bloqueado: Redirigir a completar perfil	"Tu perfil aún no tiene suficiente información para generar un CV. Completa al menos los datos básicos para continuar"
30% - 35%	 Permitir con advertencia fuerte	"Tu perfil tiene muy poca información, por lo que tu CV será bastante básico. Puedes continuar, pero te recomendamos completar más datos para obtener mejores resultados"
36% - 50%	 Permitir con sugerencia	"Agregar más información mejorará notablemente la calidad de tu CV. Aun así, ya puedes continuar con la generación."
51% - 75%	 Permitir con mensaje informativo	"Ya puedes generar un buen CV. Aun así, agregar más detalles puede hacerlo más completo y profesional."
76% - 85%	 Permitir con mensaje positivo	"Tu perfil está muy completo. Ya puedes generar un CV de alta calidad."

Rango de Completitud	Comportamiento	Mensaje UI
86% - 100%	✅ Permitir sin advertencia	"Tu perfil está listo para generar tu CV"

6.2 Validación en Backend

```
function validarPermisoGeneracionCV(puntaje: number): {

    permitido: boolean;

    nivel: 'bloqueado' | 'advertencia_fuerte' | 'sugerencia' | 'info' | 'ok';

    mensaje: string;

} {

    if (puntaje < 30) {

        return {

            permitido: false,

            nivel: 'bloqueado',

            mensaje: 'Tu perfil necesita más información antes de generar un CV.'

        };

    }

    if (puntaje >= 30 && puntaje <= 35) {

        return {

            permitido: true,

            nivel: 'advertencia_fuerte',

            mensaje: 'Tu CV será muy básico con esta información.'

```

```
};  
}
```

```
if (puntaje >= 36 && puntaje <= 50) {  
  return {  
    permitido: true,  
    nivel: 'sugerencia',  
    mensaje: 'Sería ideal completar más información para un mejor CV.'  
  };  
}
```

```
if (puntaje >= 51 && puntaje <= 75) {  
  return {  
    permitido: true,  
    nivel: 'info',  
    mensaje: 'Puedes generar un buen CV, aunque podría mejorar con más detalles.'  
  };  
}
```

```
if (puntaje >= 76 && puntaje <= 85) {  
  return {  
    permitido: true,
```

```
    nivel: 'info',

    mensaje: 'Tu perfil está muy bien. Puedes generar un CV de calidad.'

  };

}

return {

  permitido: true,

  nivel: 'ok',

  mensaje: "

};

}
```

7. Tests Unitarios Requeridos

7.1 Tests de Capa 1

```
describe('Capa 1 - Datos Básicos', () => {

  test('perfil vacío debe dar 0 puntos', () => {

    const perfil = crearPerfilVacio();

    expect(calcularCompleitudPerfil(perfil)).toBe(0);

  });

  test('solo nombre y apellido debe dar 10 puntos', () => {
```

```
const perfil = crearPerfilVacio();

perfil.informacion_personal.nombre = 'Juan';

perfil.informacion_personal.apellido = 'García';

expect(calcularCompleitudPerfil(perfil)).toBe(10);

});
```

```
test('nombre, apellido y email debe dar 15 puntos', () => {

  const perfil = crearPerfilVacio();

  perfil.informacion_personal.nombre = 'Juan';

  perfil.informacion_personal.apellido = 'García';

  perfil.informacion_personal.email_cv = 'juan@example.com';

  expect(calcularCompleitudPerfil(perfil)).toBe(15);

});
```

```
test('todos los campos de Capa 1 completos debe dar 30 puntos', () => {

  const perfil = crearPerfilVacio();

  perfil.informacion_personal = {

    nombre: 'Juan',

    apellido: 'García',

    email_cv: 'juan@example.com',

    foto_url: 'https://example.com/foto.jpg',
```

```
    profesion_perfil: 'Desarrollador',  
  
    telefono: '+123456789',  
  
    ciudad: 'Madrid',  
  
    pais: 'España'  
  
  };  
  
  expect(calcularCompleitudPerfil(perfil)).toBe(30);  
  
});  
  
});
```

7.2 Tests de Experiencia

```
describe('Experiencia Laboral', () => {  
  
  test('0 experiencias debe dar 0 puntos adicionales', () => {  
  
    const perfil = crearPerfilConCapa1Completa();  
  
    expect(calcularCompleitudPerfil(perfil)).toBe(30);  
  
  });  
  
  
  test('1 experiencia válida debe dar 15 puntos', () => {  
  
    const perfil = crearPerfilConCapa1Completa();  
  
    perfil.experiencia = [crearExperienciaValida()];  
  
    expect(calcularCompleitudPerfil(perfil)).toBe(45);  
  
  });  
  
  
  test('2 experiencias válidas debe dar 30 puntos', () => {
```

```

const perfil = crearPerfilConCapa1Completa();

perfil.experiencia = [crearExperienciaValida(), crearExperienciaValida()];

expect(calcularCompleitudPerfil(perfil)).toBe(60);

});

```

```

test('3+ experiencias válidas debe dar máximo 30 puntos', () => {

  const perfil = crearPerfilConCapa1Completa();

  perfil.experiencia = [

    crearExperienciaValida(),

    crearExperienciaValida(),

    crearExperienciaValida()

  ];

  expect(calcularCompleitudPerfil(perfil)).toBe(60);

});

});

```

7.3 Tests de Habilidades

```

describe('Habilidades', () => {

  test('0-2 habilidades debe dar 0 puntos', () => {

    const perfil = crearPerfilConCapa1Completa();

    perfil.habilidades = ['JavaScript'];

    expect(calcularPuntosHabilidades(perfil.habilidades)).toBe(0);
  });
});

```

```
});
```

```
test('3-5 habilidades debe dar 5 puntos', () => {  
  
  const perfil = crearPerfilConCapa1Completa();  
  
  perfil.habilidades = ['JavaScript', 'React', 'Node.js'];  
  
  expect(calcularPuntosHabilidades(perfil.habilidades)).toBe(5);  
  
});
```

```
test('6+ habilidades debe dar 7 puntos', () => {  
  
  const perfil = crearPerfilConCapa1Completa();  
  
  perfil.habilidades = ['JS', 'React', 'Node', 'SQL', 'Git', 'Docker'];  
  
  expect(calcularPuntosHabilidades(perfil.habilidades)).toBe(7);  
  
});
```

```
});
```

7.4 Tests de Logros

```
describe('Logros', () => {  
  
  test('0 logros debe dar 0 puntos', () => {  
  
    expect(calcularPuntosLogros([])).toBe(0);  
  
  });  
  
});
```

```
test('1-2 logros debe dar 5 puntos', () => {  
  
  expect(calcularPuntosLogros(['Logro 1'])).toBe(5);  
  
});
```

```
    expect(calcularPuntosLogros(['Logro 1', 'Logro 2'])).toBe(5);  
  });
```

```
test('3+ logros debe dar 8 puntos', () => {  
  expect(calcularPuntosLogros(['L1', 'L2', 'L3'])).toBe(8);  
  expect(calcularPuntosLogros(['L1', 'L2', 'L3', 'L4'])).toBe(8);  
});  
});
```

7.5 Tests de Eliminación

```
describe('Eliminación de Información', () => {  
  test('eliminar 1 de 2 experiencias debe restar 15 puntos', () => {  
    const perfil = crearPerfilConCapa1Completa();  
    perfil.experiencia = [crearExperienciaValida(), crearExperienciaValida()];  
    const puntajeAntes = calcularCompleitudPerfil(perfil); // 60  
  
    perfil.experiencia = [perfil.experiencia[0]];  
    const puntajeDespues = calcularCompleitudPerfil(perfil); // 45  
  
    expect(puntajeAntes - puntajeDespues).toBe(15);  
  });  
});
```

```
test('eliminar 1 de 3+ experiencias NO debe restar puntos', () => {
```

```
  const perfil = crearPerfilConCapa1Completa();
```

```
  perfil.experiencia = [
```

```
    crearExperienciaValida(),
```

```
    crearExperienciaValida(),
```

```
    crearExperienciaValida()
```

```
  ];
```

```
  const puntajeAntes = calcularCompleitudPerfil(perfil); // 60
```

```
  perfil.experiencia = perfil.experiencia.slice(0, 2);
```

```
  const puntajeDespues = calcularCompleitudPerfil(perfil); // 60
```

```
  expect(puntajeAntes).toBe(puntajeDespues);
```

```
});
```

```
test('eliminar habilidad de 6 a 5 debe restar 2 puntos', () => {
```

```
  const perfil = crearPerfilVacio();
```

```
  perfil.habilidades = ['JS', 'React', 'Node', 'SQL', 'Git', 'Docker'];
```

```
  const puntajeAntes = calcularPuntosHabilidades(perfil.habilidades); // 7
```

```
  perfil.habilidades = perfil.habilidades.slice(0, 5);
```

```
  const puntajeDespues = calcularPuntosHabilidades(perfil.habilidades); // 5
```

```
    expect(puntajeAntes - puntajeDespues).toBe(2);

  });

});
```

8. Integración con UI

8.1 Componente de Barra de Progreso

```
interface BarraCompleitudProps {

  puntaje: number;

}

function BarraCompleitud({ puntaje }: BarraCompleitudProps) {

  const { color, texto } = obtenerEstadoCompleitud(puntaje);

  return (

    <div className="w-full">

      <div className="flex justify-between mb-2">

        <span className="text-sm font-medium">Compleitud del perfil</span>

        <span className="text-sm font-semibold">{puntaje}%</span>

      </div>

      <div className="w-full bg-gray-200 rounded-full h-2">

        <div
```

```

        className={`h-2 rounded-full transition-all duration-300 ${color}`}

        style={{ width: `${puntaje}%` }}

    />

</div>

<p className="text-xs text-gray-600 mt-1">{texto}</p>

</div>

);

}

function obtenerEstadoCompleitud(puntaje: number) {

    if (puntaje < 30) {

        return {

            color: 'bg-red-500',

            texto: 'Necesitas completar más información para generar un CV'

        };

    }

    if (puntaje < 50) {

        return {

            color: 'bg-orange-500',

            texto: 'Perfil básico. Añade más detalles para mejorar tu CV'

        };

    }

```

```
if (puntaje < 75) {  
  
    return {  
  
        color: 'bg-yellow-500',  
  
        texto: 'Buen perfil. Puedes generar un CV de calidad'  
  
    };  
  
}  
  
if (puntaje < 85) {  
  
    return {  
  
        color: 'bg-blue-500',  
  
        texto: 'Muy buen perfil. Tu CV será profesional y completo'  
  
    };  
  
}  
  
return {  
  
    color: 'bg-green-500',  
  
    texto: 'Perfil excelente. Listo para generar CVs optimizados'  
  
};  
  
}
```

9. Casos Edge y Validaciones

9.1 Validación de Strings Vacíos

```
function esStringValido(str: string | null | undefined): boolean {
```

```
    return str !== null && str !== undefined && str.trim().length > 0;
}
```

9.2 Validación de Email

```
function esEmailValido(email: string): boolean {

    const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    return regex.test(email.trim());

}
```

9.3 Manejo de Datos Corruptos

```
function sanitizarPerfil(perfil: any): PerfilUsuario {

    return {

        informacion_personal: {

            nombre: perfil.informacion_personal?.nombre || "",

            apellido: perfil.informacion_personal?.apellido || "",

            foto_url: perfil.informacion_personal?.foto_url || null,

            telefono: perfil.informacion_personal?.telefono || null,

            ciudad: perfil.informacion_personal?.ciudad || null,

            pais: perfil.informacion_personal?.pais || null,

            email_cv: perfil.informacion_personal?.email_cv || "",

            profesion_perfil: perfil.informacion_personal?.profesion_perfil || null

        },

        profesiones_inferidas: Array.isArray(perfil.profesiones_inferidas)

            ? perfil.profesiones_inferidas
```

```
    : [],  
    experiencia: Array.isArray(perfil.experiencia) ? perfil.experiencia : [],  
    educacion: Array.isArray(perfil.educacion) ? perfil.educacion : [],  
    habilidades: Array.isArray(perfil.habilidades) ? perfil.habilidades : [],  
    logros: Array.isArray(perfil.logros) ? perfil.logros : [],  
    idiomas: Array.isArray(perfil.idiomas) ? perfil.idiomas : [],  
    puntaje_completitud: 0,  
    ultima_actualizacion: new Date()  
};  
}
```

10. Logging y Debugging

10.1 Log Detallado de Cálculo

```
function calcularCompleitudConLog(perfil: PerfilUsuario): {  
    puntaje: number;  
    desglose: DesglosePuntos;  
}{  
    const desglose: DesglosePuntos = {  
        capa1: {  
            nombre_apellido: 0,  
            email: 0,
```

```
    foto: 0,  
  
    profesion: 0,  
  
    contacto: 0,  
  
    subtotal: 0  
  
  },  
  
  capa2: {  
  
    experiencia: 0,  
  
    educacion: 0,  
  
    habilidades: 0,  
  
    logros: 0,  
  
    idiomas: 0,  
  
    subtotal: 0  
  
  },  
  
  total: 0  
  
};
```

```
// Calcular Capa 1
```

```
if (perfil.informacion_personal.nombre?.trim() &&  
    perfil.informacion_personal.apellido?.trim()) {  
  
  desglose.capa1.nombre_apellido = 10;  
  
}
```

```
if (perfil.informacion_personal.email_cv?.trim() &&  
    esEmailValido(perfil.informacion_personal.email_cv)) {  
    desglose.capa1.email = 5;  
}
```

```
if (perfil.informacion_personal.foto_url) {  
    desglose.capa1.foto = 5;  
}
```

```
if (perfil.informacion_personal.profesion_perfil?.trim()) {  
    desglose.capa1.profesion = 5;  
}
```

```
if (perfil.informacion_personal.telefono?.trim() &&  
    (perfil.informacion_personal.ciudad?.trim() ||  
    perfil.informacion_personal.pais?.trim())) {  
    desglose.capa1.contacto = 5;  
}
```

```
desglose.capa1.subtotal = Object.values(desglose.capa1)  
    .filter(v => typeof v === 'number')
```

```
.reduce((a, b) => a + b, 0);
```

```
// Calcular Capa 2
```

```
desglose.capa2.experiencia = calcularPuntosExperiencia(perfil.experiencia);
```

```
desglose.capa2.educacion = calcularPuntosEducacion(perfil.educacion);
```

```
desglose.capa2.habilidades = calcularPuntosHabilidades(perfil.habilidades);
```

```
desglose.capa2.logros = calcularPuntosLogros(perfil.logros);
```

```
desglose.capa2.idiomas = calcularPuntosIdiomas(perfil.idiomas);
```

```
desglose.capa2.subtotal = Object.values(desglose.capa2)
```

```
.filter(v => typeof v === 'number')
```

```
.reduce((a, b) => a + b, 0);
```

```
desglose.total = desglose.capa1.subtotal + desglose.capa2.subtotal;
```

```
console.log('Desglose de completitud:', JSON.stringify(desglose, null, 2));
```

```
return {
```

```
  puntaje: desglose.total,
```

```
  desglose
```

```
};
```

```
}
```

```
interface DesglosePuntos {  
  
  capa1: {  
  
    nombre_apellido: number;  
  
    email: number;  
  
    foto: number;  
  
    profesion: number;  
  
    contacto: number;  
  
    subtotal: number;  
  
  };  
  
  capa2: {  
  
    experiencia: number;  
  
    educacion: number;  
  
    habilidades: number;  
  
    logros: number;  
  
    idiomas: number;  
  
    subtotal: number;  
  
  };  
  
  total: number;  
  
}
```

11. Migración y Retrocompatibilidad

11.1 Script de Migración

Si ya existen perfiles en producción que usan la fórmula antigua con resumen profesional:

```
async function migrarCompleitudPerfiles() {  
  
  const perfiles = await obtenerTodosLosPerfiles();  
  
  for (const perfil of perfiles) {  
  
    const nuevosPuntos = calcularCompleitudPerfil(perfil);  
  
    await actualizarPerfil(perfil.id, {  
  
      puntaje_compleitud: nuevosPuntos,  
  
      ultima_actualizacion: new Date()  
  
    });  
  
    console.log(`Perfil ${perfil.id}: ${perfil.puntaje_compleitud} → ${nuevosPuntos}`);  
  
  }  
  
  console.log('Migración completada');  
}
```

12. Resumen de Implementación

12.1 Checklist de Implementación

- ☐ Crear interfaces TypeScript de `PerfilUsuario`, `ExperienciaLaboral`, `Educacion`, `Idioma`
- ☐ Implementar función `calcularPuntosCapa1()`
- ☐ Implementar función `calcularPuntosExperiencia()`
- ☐ Implementar función `calcularPuntosEducacion()`
- ☐ Implementar función `calcularPuntosHabilidades()`
- ☐ Implementar función `calcularPuntosLogros()`
- ☐ Implementar función `calcularPuntosIdiomas()`
- ☐ Implementar función principal `calcularCompleitudPerfil()`
- ☐ Implementar función `validarPermisoGeneracionCV()`
- ☐ Crear triggers en base de datos para recálculo automático
- ☐ Implementar componente React `BarraCompleitud`
- ☐ Escribir tests unitarios para todos los casos
- ☐ Crear script de migración si es necesario
- ☐ Implementar logging detallado para debugging
- ☐ Documentar en Swagger/OpenAPI los endpoints relacionados

12.2 Endpoints API Necesarios

// GET /api/perfil/:userId/compleitud

// Response: { puntaje: number, desglose: DesglosePuntos, permitido: boolean }

// POST /api/perfil/:userId/recalcular-compleitud

// Response: { puntaje: number }

// GET /api/perfil/:userId/validar-generacion-cv

// Response: { permitido: boolean, nivel: string, mensaje: string }

13. Notas Finales para Claude Code

1. **Prioridad máxima:** Implementar el cálculo exacto según las tablas y fórmulas de este documento.
2. **No simplificar:** Cada regla escalonada (habilidades 3-5 vs 6+, logros 1-2 vs 3+) existe por diseño de producto.
3. **Validación estricta:** Siempre usar `trim()` y validar que strings no estén vacíos antes de contabilizar.
4. **Testing exhaustivo:** Los tests unitarios deben cubrir todos los casos edge mencionados.
5. **Performance:** El cálculo debe ser $O(n)$ donde n es el número total de elementos en el perfil. No iterar múltiples veces sobre los mismos arrays.
6. **Inmutabilidad:** Nunca mutar el objeto `perfil` original durante el cálculo.
7. **Type safety:** Usar TypeScript strict mode y evitar `any`.
8. **Logging:** En desarrollo, siempre loggear el desglose completo para debugging.

Fin del documento técnico